

Teacher Step-by-Step Tutorial

Drone Vision Challenges with OpenCV, NumPy, and Class-Based Design

This document is a classroom guide for the teacher branch of the repository. It is designed to be followed live during class and includes setup, pacing, commands, lesson prompts, student checkpoints, and troubleshooting for all 8 challenges.

Repository	F:\dev\drone
Branch for class	teachers
Student branch	students
Challenge folder	F:\dev\drone\activities\challenges
Video folder	Set by TEACHER_VIDEO_DIR in video_config.py or DRONE_VIDEO_DIR

1. Before Students Arrive

Use the following checklist before the lesson starts. This prevents most class-time problems.

- Open a terminal in F:\dev\drone.
- Switch to the teachers branch so the reference solutions are available.
- Activate the conda environment cardd-paper-demo.
- Set PYTHONPATH to F:\dev\drone\src.
- Set DRONE_VIDEO_DIR or edit video_config.py so the scripts can find the videos.
- Run one teacher solution before class to confirm the machine can open OpenCV windows.
- Open the starter file for the first challenge in your editor.

```
cd F:\dev\drone
git switch teachers
conda activate cardd-paper-demo
$env:PYTHONPATH = "F:\dev\drone\src"
$env:DRONE_VIDEO_DIR = "D:\course_materials\drone_videos"
```

2. Suggested Teaching Rhythm

- Begin each challenge by showing the teacher solution briefly so students know the target behavior.
- Then switch to the student starter file and focus on the TODOs only.
- Let students work in pairs for 10 to 20 minutes depending on the challenge.
- Pause the room when a common bug appears and explain it once for everyone.
- End each challenge with a 2 to 5 minute debrief that connects the code to future ROS 2 ideas.

3. How to Use the Branches

- Use the teachers branch on the classroom machine when you need reference solutions.
- Distribute the students branch to learners so they receive only the starter files.
- If you project code in class, make sure you are displaying the correct branch before opening a file.

```
git switch teachers
git switch students
```

Challenge 1: Camera Viewer Class

Recommended time: 30 minutes

Video or input: Bluemlisalphutte Flyover.mp4

Files: activities/challenges/challenge_01_camera_student.py, activities/challenges/challenge_01_camera_teacher.py

Teacher goal: Help students understand that a drone camera pipeline is a loop over frames plus metadata such as time and frame index.

Before class

- Confirm the teacher branch is checked out.
- Confirm DRONE_VIDEO_DIR points to the folder that contains Bluemlisalphutte Flyover.mp4.
- Open the student starter file before class so the TODOs are visible.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_01_camera_teacher.py
```

Suggested explanation to students

- Today we are treating a prerecorded drone video as if it were the live camera of a real robot.
- Every later perception and control module depends on a reliable frame loop.
- In ROS 2 this would become a camera node publishing images on a topic.

Step-by-step classroom flow

1. Ask students to find where the video should be opened.
2. Ask them to check whether the video opened successfully.
3. Ask them to read one frame at a time and stop when the video ends.
4. Ask them to resize the frame and overlay frame number and timestamp.
5. Ask them to display the frame and close the window with q.

What to check while students work

- The video window opens.
- The frame count changes.
- The timestamp increases steadily.
- The script exits cleanly when q is pressed.

Common issues to watch for

- Students forget `str(video_path)` when opening `cv2.VideoCapture`.
- Students do not stop when `cap.read()` fails at the end of the file.
- Students forget `cap.release()` or `cv2.destroyAllWindows()`.

Closing discussion

- Ask: what does each loop iteration represent?
- Ask: why is FPS needed to estimate timestamp?
- Connect this to the future ROS 2 camera publisher.

Challenge 2: Frame Packet Class

Recommended time: 35 minutes

Video or input: Stockflue Flyaround.mp4

Files: activities/challenges/challenge_02_packet_student.py, activities/challenges/challenge_02_packet_teacher.py

Teacher goal: Introduce structured frame data and show that NumPy can turn raw images into measurements.

Before class

- Confirm Stockflue Flyaround.mp4 exists in the selected video directory.
- Remind students that image arrays are numeric data, not just pictures.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_02_packet_teacher.py
```

Suggested explanation to students

- A robotics system rarely passes around raw images alone.
- We often package the image together with timing, index, and derived measurements.
- This FramePacket dataclass is the classroom version of a future message object.

Step-by-step classroom flow

1. Ask students to validate the camera stream in open().
2. Ask them to return None when the video ends.
3. Ask them to convert the frame to grayscale.
4. Ask them to compute average brightness using np.mean.
5. Ask them to build and return a FramePacket object.

What to check while students work

- Brightness values appear on screen.
- Students use a dataclass rather than a loose tuple.
- The program stops cleanly when the video ends.

Common issues to watch for

- Students compute np.mean on the original color frame and get confusing results.
- Students forget to increment frame_index.
- Students forget to return the FramePacket object.

Closing discussion

- Ask: why is brightness useful as a simple derived feature?
- Ask: why is a dataclass helpful for larger robotics systems?

Challenge 3: Manual Pilot Class

Recommended time: 40 minutes

Video or input: Bluemlisalphutte Flyover.mp4

Files: activities/challenges/challenge_03_teleop_student.py, activities/challenges/challenge_03_teleop_teacher.py

Teacher goal: Connect the video to manual flight concepts: roll, pitch, yaw, and throttle.

Before class

- Review the meaning of roll, pitch, yaw, and throttle before coding.
- Tell students that these are simulated commands, not real aircraft commands.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_03_teleop_teacher.py
```

Suggested explanation to students

- Now we add the idea of piloting.
- The camera feed is still prerecorded, but the control layer is live and interactive.
- Later, this same idea will become a ROS 2 command publisher.

Step-by-step classroom flow

1. Ask students to map a/d to roll.
2. Ask students to map w/s to pitch.
3. Ask students to map j/l to yaw and i/k to throttle.
4. Ask them to add a space-bar reset.
5. Ask them to clamp every axis to the range [-1, 1].
6. Ask them to draw all four values on the frame.

What to check while students work

- Each key changes exactly one control axis.
- The numbers on the HUD respond immediately.
- The emergency stop resets all values.

Common issues to watch for

- Students mix up yaw and roll.
- Students let the values grow without bounds.
- Students update commands before reading the key correctly.

Closing discussion

- Ask students which axis would rotate the drone left-right versus turn its heading.
- Connect the HUD to the idea of telemetry and operator feedback.

Challenge 4: Telemetry Parser Class

Recommended time: 45 minutes

Video or input: DJI_0574.MP4 and DJI_0574.SRT

Files: activities/challenges/challenge_04_telemetry_student.py, activities/challenges/challenge_04_telemetry_teacher.py

Teacher goal: Show that the drone has more sensors than just the camera and that multiple data streams must be aligned by time.

Before class

- Verify that both DJI_0574.MP4 and DJI_0574.SRT are present in the chosen folder.
- Explain that the SRT file contains time-stamped telemetry text.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_04_telemetry_teacher.py
```

Suggested explanation to students

- Real robots rarely operate from vision alone.
- Here we align a camera stream with a telemetry stream.
- This is a small step toward sensor fusion thinking.

Step-by-step classroom flow

1. Ask students to locate the matching SRT file using `video_path.with_suffix('.SRT')`.
2. Ask them to split the file into subtitle blocks.
3. Ask them to parse the start time of each block.
4. Ask them to extract GPS and barometer values with regular expressions.
5. Ask them to return the latest sample at or before the current video time.
6. Ask them to draw telemetry values on the frame.

What to check while students work

- Telemetry appears only for the DJI video with SRT data.
- Values update as the video time increases.
- Students can explain why a latest-sample lookup is needed.

Common issues to watch for

- Students reverse latitude and longitude.
- Students assume every video has a matching SRT.
- Students use the wrong subtitle line for the timestamp.

Closing discussion

- Ask: why do we align by time rather than frame index alone?
- Ask: what other sensors might a real drone publish?

Challenge 5: Virtual Target Tracker Class

Recommended time: 50 minutes

Video or input: DJI_0596.MP4

Files: activities/challenges/challenge_05_target_student.py, activities/challenges/challenge_05_target_teacher.py

Teacher goal: Build a complete perception step: target creation, segmentation, centroid extraction, and visualization.

Before class

- Explain why a virtual target makes the exercise reliable for every student.
- Review the difference between BGR and HSV color spaces.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_05_target_teacher.py
```

Suggested explanation to students

- We now create the target that the drone should eventually track.
- First we create a clean perception problem, then we solve it.
- The output of this stage is no longer an image alone; it is a measurement of target position.

Step-by-step classroom flow

1. Ask students to draw a moving red circle onto each frame.
2. Ask them to convert the image from BGR to HSV.
3. Ask them to threshold the red target color.
4. Ask them to use np.where to find mask coordinates.
5. Ask them to compute centroid and area.
6. Ask them to draw the centroid and display the mask.

What to check while students work

- A red target is visible on the video.
- The mask window isolates the target.
- The centroid marker follows the target.

Common issues to watch for

- Students use only one red hue range and miss some pixels.
- Students forget to handle the no-detection case.
- Students draw on the mask instead of the video frame.

Closing discussion

- Ask: why is HSV often easier for color segmentation?
- Ask: what information would a controller need from this detector?

Challenge 6: Proportional Controller Class

Recommended time: 35 minutes

Video or input: Uses example detections rather than a video loop

Files: activities/challenges/challenge_06_controller_student.py, activities/challenges/challenge_06_controller_teacher.py

Teacher goal: Translate perception error into corrective action with a proportional controller.

Before class

- Review the idea of error: measured position minus desired position.
- Tell students that the center of the image is the desired target position.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_06_controller_teacher.py
```

Suggested explanation to students

- The detector gives us a target position; the controller turns that into action.
- Today we start with a proportional controller because it is easy to understand and debug.
- The sign of the feedback matters more than the exact gain at first.

Step-by-step classroom flow

1. Ask students to implement clamp().
2. Ask them to define behavior when the target is not found.
3. Ask them to convert error_x into roll and yaw corrections.
4. Ask them to convert error_y into throttle correction.
5. Ask them to add forward pitch only when the target area is small.

What to check while students work

- Outputs remain inside the defined limits.
- The command signs point in the corrective direction.
- Students can explain search behavior when the target is missing.

Common issues to watch for

- Students implement positive feedback instead of negative feedback.
- Students forget to clamp one of the outputs.
- Students ignore the missing-target case.

Closing discussion

- Ask: what happens if the gains are too large?
- Ask: why does the controller need normalized error rather than raw pixel values?

Challenge 7: Mission Manager Class

Recommended time: 35 minutes

Video or input: Uses a target-state sequence rather than a video loop

Files: activities/challenges/challenge_07_mission_student.py, activities/challenges/challenge_07_mission_teacher.py

Teacher goal: Separate high-level mission decisions from low-level control.

Before class

- Introduce SEARCH, ALIGN, HOLD, and LAND as mission-level states.
- Explain that state machines help us reason about autonomy step by step.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_07_mission_teacher.py
```

Suggested explanation to students

- A controller answers how to move; a mission manager answers what mode the robot should be in.
- This is the first layer of autonomy above direct control.
- We want state transitions that are explainable and stable.

Step-by-step classroom flow

1. Ask students to return a search command when the target is missing.
2. Ask them to switch to ALIGN when the target is visible but off-center.
3. Ask them to count centered steps in HOLD.
4. Ask them to transition to LAND only after a stable hold period.

What to check while students work

- The printed state progression makes sense.
- The hold counter prevents immediate landing.
- Students separate state logic from controller math.

Common issues to watch for

- Students forget to reset hold_counter when the target is lost.
- Students jump to LAND too early.
- Students mix target-centering logic with state transitions in one unreadable block.

Closing discussion

- Ask: why is state memory important?
- Ask: what other drone mission states could exist in a real system?

Challenge 8: Full Pipeline Integration

Recommended time: 60 minutes

Video or input: DJI_0790.MOV

Files: activities/challenges/challenge_08_integration_student.py,
activities/challenges/challenge_08_integration_teacher.py

Teacher goal: Bring together sensing, perception, control, and visualization into one coherent mock drone pipeline.

Before class

- Tell students this is the capstone challenge of the sequence.
- Ask them to reuse ideas from the earlier challenges rather than invent everything again.

Run this demo first

```
python F:\dev\drone\activities\challenges\challenge_08_integration_teacher.py
```

Suggested explanation to students

- Now we combine the building blocks into one mini autonomy stack.
- Even without ROS 2, we already have the key architecture: sensing, perception, control, and display.
- The purpose of this challenge is integration, not novelty.

Step-by-step classroom flow

1. Ask students to add the virtual target to each frame.
2. Ask them to detect the target and compute normalized error.
3. Ask them to compute roll, pitch, yaw, and throttle from the detection.
4. Ask them to draw a complete HUD with crosshairs, target, and command values.

What to check while students work

- The target is detected consistently.
- The command values react to target motion.
- The HUD clearly shows the state of the pipeline.
- Students can describe data flow from image to action.

Common issues to watch for

- Students mix raw pixel offsets with normalized error.
- Students forget to compute commands when detection is missing.
- Students draw too much on the frame and make the display hard to read.

Closing discussion

- Ask students which parts would become separate ROS 2 nodes.
- Ask them which topics they would define between those nodes.
- Close by reviewing the full path: camera -> perception -> controller -> mission.

Final Wrap-Up Discussion

- Ask students to name the main layers they built: camera, message, control, telemetry, perception, mission, integration.
- Ask which parts would become ROS 2 nodes in the future.
- Ask what messages or topics would connect those nodes.
- Ask what would need to change when moving from prerecorded video to a real drone camera.

Quick Troubleshooting Reference

- If no window opens, verify the conda environment and OpenCV installation.
- If the video does not load, verify `DRONE_VIDEO_DIR` and the exact filename.
- If telemetry does not appear, confirm the matching SRT file is present.
- If imports fail, set `PYTHONPATH` to `F:\dev\drone\src` before running examples from the repo root.
- If a student gets lost, point them to the next TODO only rather than explaining the whole solution.